

SGLang-Omni 每日 PR 学习日报

报告日期：2026-06-13 · 仓库：sgl-project/sglang-omni · 范围：过去 24 小时合并 (merged) 的 PR · 讲解深度优先

主线主题：MOSS-TTS-Local 状态池三部曲

共 8 个 merged PR

深讲 4 · 简述 4

面向：推理基础设施零基础读者

当日概览

8

过去 24h 合并 PR

3

核心架构 / 状态池化

2

Higgs TTS 优化

3

硬件 / 后端 / CI

今天合并的 8 个 PR 里，真正的主线是 MOSS-TTS-Local v1.5 这条多阶段语音合成 (TTS) 推理路径的“状态池化 (state-pool)”重构三部曲 (#745 → #759 → #758)。它们把“每一步都要用 Python 对象临时拼出来”的解码状态，搬进常驻 GPU 的、按行 (per-row) 索引的张量缓冲区里，为 CUDA Graph 捕获和 async 解码 (一步前瞻) 铺路。其余的是 Higgs TTS 的缓存 / CLI 打磨 (#713、#700) 和基础设施 (Blackwell 适配 #763、后端升级 #698、CI 镜像 #764)。

下面按“讲透优先于覆盖广”的原则：前 4 个 PR 逐块拆代码 + 补全所有背景概念；后 4 个简明说明动机与改动。

#	标题	主题	一句话
745	MOSS-TTS-Local v1.5 解码状态池	架构	把每请求解码状态搬进常驻 GPU 行索引缓冲区，输出改走“日志 (journal)”，行为零变化
759	池化 frame 启动状态	性能	把 generation_steps、重复惩罚历史也搬进池子，一次向量化 gather；无惩罚路径保持可图化
758	async 解码 (一步前瞻)	性能 架构	在 Local 路径打开“发射/收尾”分离的一步前瞻，含 bs=1 停止符的设备快照修复
713		TTS	

	Higgs 参考音频缓存键		用解码后的波形内容做参考码缓存键，让相同音频的不同 URL 也能命中缓存
700	Higgs decode-mode CLI	TTS	把 --async-decode 换成更清晰的 --decode-mode async sync
763	Blackwell GPU 适配	基础设施	按 SM 版本自动为 Blackwell(SM100+) 设置 FlashInfer/注意力后端兜底
698	升级钉死的 SGLang 后端	基础设施	升级 SGLang 依赖并适配多模型组件到新 API
* 764	CI PyPI 镜像换阿里云	基础设施	清华镜像缺 CUDA13 包导致 CI 失败，切到阿里云镜像

PR 详解（卡片可折叠）

提示：网页中点击每张卡片标题可折叠/展开；PDF 版默认全部展开以便长期保存阅读。

MOSS-TTS-Local v1.5 解码状态池 (state-pool)

作者 JiaxinD · 已合并 · #745 · 15 文件 · 核心 : state_pool.py / model_runner.py / radix_hash.py

概念 : SGLang 推理引擎的几个核心组件

它是什么 → SGLang 是一个高性能大模型推理引擎。一条请求进来，会流过几个分工明确的组件：Tokenizer Manager (把文字切成 token id) → Scheduler 调度器 (决定哪些请求这一步一起跑、组成一个 batch) → Model Runner 模型执行器 (真正在 GPU 上前向计算、采样出下一个 token) → Detokenizer (把 token 还原成文字/音频)。其中还有 Attention Backend (注意力算子的具体实现，如 FlashInfer、Triton)。

为什么需要它 → 推理服务要同时服务很多用户、还要榨干 GPU。把“切词、排队、算、还原”拆成流水线组件，每个组件可以独立优化、独立扩展，才能做到高吞吐。

在这个 PR 里怎么用 → sglang-omni 是 SGLang 的“多模态/全能 (Omni)”分支，在这套组件之上加了语音、图像等模态。今天的 PR 几乎都在改 Model Runner 这一层 (具体是 MOSS-TTS-Local 这个 TTS 模型的 runner) 和 Scheduler。

· 类比：把它想成一条餐厅后厨流水线：点单员(tokenizer)→ 排菜的(scheduler)→ 炒菜的(model runner)→ 装盘的(detokenizer)。

概念 : Continuous Batching (连续批处理)

它是什么 → 一种调度策略：不等一整批请求都到齐/都结束，而是“谁好了就让谁加入这一步的计算、谁完成了就立刻让出位置”，每一步动态重组 batch。

为什么需要它 → 传统静态批处理要等最慢的请求跑完才换下一批，GPU 大量时间在空转。连续批处理让 GPU 几乎一直满载。

在这个 PR 里怎么用 → 正因为 batch 每步都在变 (请求随时进/出/被回收)， “每请求的解码状态”必须能稳定地跟着请求走——这正是 #745 状态池要解决的根本问题。

类比：像公交车不等满员发车，而是到站就上下客、一直在跑。

概念 : KV Cache 与 Radix Attention

它是什么 → KV Cache : Transformer 每生成一个 token，都要用到前面所有 token 的 Key/Value 向量。把它们缓存起来避免重算，就是 KV cache，它占用大量显存。Radix Attention 是 SGLang 的招牌：用一棵基数树 (radix tree) 按 token 前缀共享 KV cache，相同前缀的请求复用同一份缓存。

为什么需要它 → 重算 KV 极其昂贵；不同请求/同一请求的不同步常有公共前缀，能共享就能省显存、提吞吐。

在这个 PR 里怎么用 → #745 把每帧的“radix 缓存键”从 CPU 端 blake2b 哈希换成了可被 CUDA Graph 捕获的 GPU 多项式哈希，去掉了每帧一次 GPU→CPU 同步 (见该 PR 第 3 节)。

类比：像图书馆把相同开头的书号归到同一书架，省地方也好找。

概念：CUDA Graph (CUDA 图)

它是什么 → 把一串固定的 GPU 操作“录制”成一张图，之后整张图一次性重放 (replay)，省掉 CPU 一条条发指令 (kernel launch) 的开销。

为什么需要它 → 解码阶段每步的 GPU 计算量很小，CPU 发指令的固定开销反而成了瓶颈。录成图重放能把这部分开销几乎清零。

在这个 PR 里怎么用 → 但 CUDA Graph 有个硬约束：被捕获的操作必须形状固定、地址固定、不能有依赖数据的分支、不能有 GPU↔CPU 同步。今天三个 MOSS PR 的所有设计 (常驻缓冲区、行索引、GPU 哈希、无惩罚路径保持图化) 都是在迁就这条约束。

类比：像把一段固定舞步预先编排好，演出时一键播放，而不是每次现场喊口令。

PR 概览

- 标题：[Refactor] MOSS-TTS-Local v1.5 decode-state pool (#735)
- 编号 / 作者 / 状态：#745 · JiaxinD · 已合并 merged
- 链接：github.com/sgl-project/sglang-omni/pull/745

一句话：把 MOSS-TTS-Local 解码时“每请求、对下一步关键”的状态，从分散的 Python 对象搬进一个常驻 GPU、按行索引的“状态池”，输出收集改走一份每步独立的“日志 (journal)”，整个改动逐位 (bit-identical) 不改变输出 (只有一处披露过的 bug 修复例外)。

它解决了什么问题

MOSS-TTS-Local 是一个 TTS (文字转语音) 模型的“本地 Transformer”解码路径。它每生成一帧音频，都需要一批“下一步要用的状态”，比如：

- 反馈嵌入 (feedback embedding)：上一帧的输出要变成下一帧的输入向量；
- 采样参数 + 随机种子：温度、top-p、top-k、seed (请求级别，整个请求不变)；
- 本步产出的帧：要在“下一步”被收集成最终音频。

改之前，这些状态分别躺在三个“临时容器”里：每个请求对象上挂一个 `pending_feedback_queue` 双端队列、一个组合键的 `_param_cache` 单槽缓存、以及 runner 全局的 `_pending_rows` 单槽。

问题：在连续批处理下 batch 每步都在重组，请求会被“撤回 (retraction)”，恢复时池中要加入一些挂在 Python 对象上的零散状态，既难保证 CUDA Graph 捕获所需的地址固定，也难固定，又给后续要做的 async 解码 (#734, #758) 埋了坑。说白了，状态大散乱，没法上 CUDA Graph，也没法做一步前推。

具体做了什么改动

核心是新增一个文件 `state_pool.py`，定义 `MossTTSLocalDecodeStatePool` (状态池) 和 `MossTTSLocalDecodeJournal` (日志)。涉及核心文件：

- `models/moss_tts_local/state_pool.py` (+175, 新文件) — 池子 + 日志
- `models/moss_tts_local/model_runner.py` (+102/-87) — gather/scatter 改造、GPU 哈希、chunked 行处理
- `models/moss_tts_local/radix_hash.py` (+87, 新文件) — 可捕获的 GPU 多项式哈希

- `model_runner/base.py` (+14/-1) — 新增通用钩子 `finalize_skip_rids`
- `radix_hash.py` 设计文档 + 6 个测试文件 (含 622 行池子测试)

(a) 池子怎么开：尺寸“蹭”现有权重表，地址锁死一辈子

```
def __init__(self, model):
    weight = model._decode_input_embedding.weight
    # P = max_running_requests + 1; +1 是预留的“填充行(padding row)”
    self.num_rows = int(weight.shape[0]) + 1
    self.padding_row = self.num_rows - 1
    self.hidden_size = int(weight.shape[1])
    self.device = weight.device
    self.dtype = weight.dtype
    # 下一帧的反馈嵌入; dtype 用 bf16, 与 staging 表一致,
    # 这样 before_decode 里的 gather 就是一次纯拷贝 (为 #736 铺路)
    self.feedback_embeds = torch.zeros(
        self.num_rows, self.hidden_size,
        device=self.device, dtype=self.dtype,
    )
```

逐块解释 (假设你没读过这段) :

- 不写死行数：池子大小直接读现有的 `_decode_input_embedding.weight` (解码输入嵌入表) 的形状。它有几行就代表能并发几个请求，于是池子自动跟着 `max_running_requests` 走。
- `num_rows = shape[0] + 1`：每个并发请求一行，额外多一行当“填充行”，索引 `padding_row`。这一行专门留给“这个槽位现在不是真实请求”的情况 (为将来上 CUDA Graph 做准备：#736)。
- `device/dtype` 全部抄权重表：保证之后的 `gather/scatter` 都是同设备同精度，无需类型转换，纯内存拷贝。
- `feedback_embeds` 是一个 `[P, hidden]` 的 GPU 大缓冲区，在模型初始化时 (任何图捕获之前) 一次性分配，于是它的显存地址在整个进程生命周期固定不变——这正是 CUDA Graph 能捕获它的前提。

另外 7 个采样相关张量 (text/audio 的 temperature、top-p、top-k 和 seeds) 也都是 `[P]` 形状，在请求“拿到行”时写一次。

(b) 收尾时把反馈嵌入“撒 (scatter)”进池子 + 写日志

```
# 只处理“真正会发帧”的行（过滤掉 chunked-prefill 的中间块）
emit_indices = [i for i, r in enumerate(requests)
                 if not self._is_chunked_request(r)]
if not emit_indices:
    return
emit_index_t = torch.tensor(emit_indices, dtype=torch.long, device=rows.device)
emit_row_t = row_t[emit_index_t.to(device=row_t.device)] # 这些请求各自的行
emit_embeds = embeds.index_select(0, emit_index_t.to(device=embeds.device))
# scatter: 把每个请求“下一步的反馈嵌入”写进它固定的池子行
pool.feedback_embeds[emit_row_t] = emit_embeds.detach().to(
    device=pool.feedback_embeds.device, dtype=pool.feedback_embeds.dtype)
# 日志: 记录这步发了帧的 rid、池子行、以及本步私有的 rows 张量, 留给“下一步”收集
result.moss_journal = MossTTSLocalDecodeJournal(
    rids=[requests[i].request_id for i in emit_indices],
    pool_rows=[pool_rows[i] for i in emit_indices],
    rows=rows.index_select(0, emit_index_t),
)
```

scatter (散射) 就是“把一批值按行号写到大缓冲区的对应行”。这里把每个请求新算出的反馈嵌入, 写回它在池子里那一行——下一步直接按行号取就行。

(c) 下一步开头把反馈嵌入“聚 (gather)”回来

```
pool_rows = [pool.acquire_row(r.request_id) for r in requests] # 幂等: 已有行就取
row_tensor = torch.tensor(pool_rows, dtype=torch.long, device=weight.device)
with torch.no_grad():
    # gather: 按本步 batch 顺序, 从池子里取出每个请求存好的反馈嵌入
    weight[:batch_size].copy_(pool.feedback_embeds[row_tensor])
```

gather (聚集) 是 scatter 的逆操作: 按行号把分散的行取出、拼成本步 *batch* 的顺序, 直接拷进解码输入嵌入表。因为同精度 (bf16), 这是一次纯拷贝、没有 GPU→CPU 同步, 对 CUDA Graph 友好。这套 gather/scatter 完全取代了老代码里 `pending_feedback_queue.popleft()` + `torch.stack` 的逐请求拼装。

(d) 为什么要单独一份“日志 (journal)”?

关键洞察: 池子的行会被同一个请求在下一步覆盖写。所以池子不能同时承担“这步产出、下步才收集”的输出数据——否则下一步一覆盖, 上一步的产出就没了。于是 journal 是一份每步独立、不可变的小快照 (*rids* + 池子行 + 本步私有 *rows* 张量), 挂在 `result` 上; 其中 *rids* 纯粹用来在收集时做一次“日志和 *batch* 对得上吗”的对齐断言。

(e) 一处披露的 bug 修复 + 一个通用钩子

```
def finalize_skip_rids(self, scheduler_output) -> set:
    """非最终的 chunked-prefill 行, 这一步不能推进 generation_steps."""
    return {r.request_id for r in scheduler_output.requests
            if self._is_chunked_request(r)}
```

采样位置是按 `position = generation_steps * num_channels + channel` 算的。如果一个被“分块预填充 (chunked prefill)”切成 K 块 of 的请求, 每块都让 `generation_steps` +1, 就会把最终块第一帧的采样位置整体挪偏, 破坏“逐位一致”。这个在 `base.py` 里新增的模型无关钩子

`finalize_skip_rids` (默认空集合) 让模型可以声明“这些请求 id 本步别推进步数”, 从而只让最终块推进一次。

概念：状态池化 (State Pooling) 这一设计模式

它是什么 → 把“本来按对象 / 队列零散存放、每步重建”的状态, 预先分配成一块固定大小、按整数行号索引的连续缓冲区 (这里是 GPU 张量), 用“拿行 / 还行”的方式管理生命周期。

为什么需要它 → 零散对象在热路径上反复创建/拷贝/拼装, 既慢又不满足 CUDA Graph 的“地址固定、形状固定、无同步”要求。池化后, 访问变成一次向量化的 gather/scatter, 地址恒定, 可被图捕获。

在这个 PR 里怎么用 → 本 PR 把反馈嵌入、采样参数、种子全部池化为 `[P, ...]` 张量; 行通过 `acquire_row` / `release_row` 管理 (填充行永不进空闲表)。这是后续 #759、#758 的地基。

类比：像把每个客人的随身物品从“到处乱放”改成“统一寄存柜+号码牌”：要用就报号码一次性取出, 柜子地址永远不变。

概念：Chunked Prefill (分块预填充)

它是什么 → “预填充 (prefill)”是把请求的整段提示词一次性喂进模型、建好 KV cache 的阶段。当提示很长、显存/计算放不下时, 把它切成几块 (chunk) 分多步喂, 就是分块预填充。

为什么需要它 → 长上下文一次性预填充会撑爆显存、还会让其它请求长时间排队; 切块能平滑显存峰值、让长短请求公平共享 GPU。

在这个 PR 里怎么用 → 切块带来一个隐患：中间块也会进入输出处理流程。本 PR 用 `is_chunked>0` 守卫跳过中间块发帧, 并用钩子避免中间块虚假推进采样步数, 保证切块路径与不切块路径逐位一致。

类比：像把一篇长文分页录入, 但只在最后一页录完才算“真正写下了第一个字”, 中间翻页不算数。

概念：可捕获的 GPU 多项式哈希（替换 host 端 blake2b）

它是什么 → 把每帧若干通道的 token id，用霍纳法（Horner）做一个定系数多项式哈希，全程 int64 张量算子（乘、加、取模），结果落在 `[0, 151643)`。

为什么需要它 → 老做法是在 CPU 上用 blake2b 哈希算 radix 键，每帧都要把数据从 GPU 搬到 CPU（一次 D2H 同步），在解码热路径上很伤。

在这个 PR 里怎么用 → 新哈希循环次数固定（通道数固定）、无数据依赖分支、无 GPU↔CPU 同步，所以能被 CUDA Graph 捕获，去掉每帧一次同步；并用 `torch.where(next_text==end_id, ...)` 保留 EOS 原值，确保停止判定不受影响。它只决定 radix 缓存命中/未命中，绝不影响采样出的 token。

类比：像给每帧算一个“指纹号”，以前要寄到外地鉴定（搬 CPU），现在车间内部就能秒出（留在 GPU）。

我能学到什么 / 延伸思考

- 可复用工程思想：池化 + 行索引 + 向量化 gather/scatter，是把“逐对象热路径”改造成“硬件友好批量操作”的通用套路，不只用于推理。
- “行为中立 + 逐位验证”的重构纪律：用固定种子门禁（SO gate）证明输出 bit-identical，把“纯重组”和“披露的行为变化”严格分开，是大型重构可信落地的范本。
- 想深入可前置了解：CUDA Graph 的捕获约束、PyTorch 的 `index_select` 高级索引，以及 Transformer 解码阶段（decode vs prefill）的区别。

PR #759 · 性能

池化 frame 启动状态 (generation_steps + 重复惩罚历史)

作者 Hayden727 · 已合并 · #759 · Closes #757 · 8 文件

PR 概览

- 标题：[Perf] Pool MOSS-TTS Local frame launch state
- 编号 / 作者 / 状态：#759 · Hayden727 · 已合并 (Closes #757)
- 链接：[pull/759](#)

一句话：在 #745 的池子基础上，把剩下还在每帧用 Python 重建的“启动状态”——generation_steps、以及音频重复惩罚的历史——也搬进池子，改成一次向量化 gather；并且让“无重复惩罚”的常见路径继续走 CUDA Graph。

它解决了什么问题

#745 把反馈嵌入和采样参数池化了，但 `_collect_frame` (收集帧) 这个热函数里还残留着每帧都重做的“启动准备”：

- 循环遍历请求去 acquire 行号；
- 用 Python list 拼出 `gen_steps` 张量；
- 从 `output_rows` 里逐请求 gather 出“重复惩罚历史”；
- 从请求对象上逐个读 `generation_steps`。

痛点：这些“每帧一次的 Python 小循环 + 临时张量”是 GPU 启动前的纯 CPU 开销。在大 batch 下累积明显（实测 bs=64 时 launch 准备从 79.77ps 降到 44.17ps）。更重要的是，重复惩罚的历史是“数据依赖、形状不定”的，正好是 CUDA Graph 的天敌——必须把它整理成“常驻、定形”的结构，才能让常见路径保持图化。

具体做了什么改动

核心文件：`state_pool.py` (+141) 新增常驻缓冲区与方法、`model_runner.py` (+191/-100) 改写收集逻辑、`base.py` (+18/-18) 新增提交钩子。

(a) 新增 4 个常驻池子缓冲区

```
self.generation_steps = torch.zeros(self.num_rows, device=self.device, dtype=
self.sampling_steps = torch.zeros(self.num_rows, device=self.device, dtype=
self.audio_repetition_penalty = torch.zeros(self.num_rows, device=self.device
# 关键新结构：每行 × 每通道 × 整个音频词表 的布尔“出现过”掩码
self.audio_token_presence = torch.zeros(
    self.num_rows, self.n_vq, self.audio_vocab_size,
    device=self.device, dtype=torch.bool)
```

- generation_steps：每行一个 int64，记“已提交”的解码步数（调度器推进的真值）。
- sampling_steps：独立的“发射位置”计数。在一步前瞻（#758）下它会领先 generation_steps，所以单独存、并对 generation_steps 取 max 做下限。
- audio_repetition_penalty：每行的惩罚系数，**1.0** 表示关闭。
- audio_token_presence：最妙的一个。形状 **[P, n_vq, audio_vocab]** 的布尔掩码，**[行, 通道, token]=True** 表示该请求在该通道生成过这个音频码。它把“每步重新 gather+去重历史张量”换成了定形、O(1) 可索引的掩码。

(b) 一次 gather，全步复用：把 row_t 缓存到 forward_batch

```
# 这一步只在写解码输入嵌入时做一次：拿行、写静态参数、判断有无惩罚
row_tensor, pool_rows, has_pen = pool.prepare_active_rows(requests)
with torch.no_grad():
    weight[:batch_size].copy_(pool.feedback_embeds[row_tensor])
forward_batch.moss_pool_row_t = row_tensor # 缓存，供后续复用
forward_batch.moss_pool_rows = pool_rows
forward_batch.moss_has_audio_repetition_penalty = has_pen
# ... 后面 _run_frame_decode 直接复用 row_t，不再重建：
gen_steps = torch.maximum(pool.sampling_steps[row_t], pool.generation_steps[r
rep_penalties = pool.audio_repetition_penalty[row_t]
```

老代码用 Python 循环 **[float(d.audio_repetition_penalty) for d in datas]** 逐请求取值；现在 **gen_steps** 和 **rep_penalties** 都是一次向量化 gather（按行号张量索引）。**torch.maximum(sampling_steps, generation_steps)** 就是“发射位置不低于已提交步数”的下限逻辑。

(c) 把“已提交步数”镜像回池子的批量钩子

```
def commit_generation_steps(self, row_t, generation_steps):
    """把这一批已提交的步数，一次写进对应池子行。"""
    if row_t.numel() == 0:
        return
    steps = generation_steps.to(device=self.device, dtype=torch.int64)
    row_t = row_t.to(device=self.device, dtype=torch.long)
    self.generation_steps[row_t] = steps
    self.sampling_steps[row_t] = torch.maximum(self.sampling_steps[row_t], steps)
```

`base.py` 的 `_finalize` 里，每个未跳过的请求把 `data.generation_steps` 自增（这是真值提交），再把（请求，新步数）收集起来，一次性触发 `on_generation_steps_advanced` 钩子。MOSS 的实现走快路径：若 batch 上缓存的 `row_t` 长度匹配，就一把 `commit_generation_steps` 全行写完。`sampling_steps` 取 max 确保它永不回退到已提交步数以下（维持前瞻不变式）。

(d) 无惩罚走 CUDA Graph，有惩罚走 eager 的掩码版

```
# 是否走图：只要这批里没有任何一行开了惩罚，且 batch 不超过捕获上限
use_graph = (not has_audio_repetition_penalty) and (batch_size <= frame_graph)

def _apply_audio_repetition_penalty_mask(logits, token_presence, penalties):
    penalties = penalties.to(device=logits.device, dtype=logits.dtype)
    active = token_presence.to(torch.bool) & (penalties != 1.0).unsqueeze(-1)
    penalties = penalties.unsqueeze(-1)
    # v1.5 规则：正 logit 除以惩罚、负 logit 乘以惩罚（都向不利方向推）
    penalized = torch.where(logits < 0, logits * penalties, logits / penalties)
    logits.copy_(torch.where(active, penalized, logits)) # 只在 active 处改写
```

关键设计：`has_audio_repetition_penalty` 是一个极便宜的 host 端布尔判断（看这批里有没有任何一行的惩罚 $\neq 1.0$ ）。

- 没有任何惩罚（绝大多数请求）→ `use_graph=True`，整帧走捕获好的 CUDA Graph 重放，惩罚相关的“数据依赖 gather + torch.where”根本不执行，自然不会破坏图捕获。
- 有惩罚 → 退回 eager，但用 `audio_token_presence` 掩码做一次定形张量运算，而不是逐请求 Python 历史 gather。混合 batch 里没开惩罚的行被 `penalties!=1.0` 自动屏蔽。

撤回-恢复的正确性也照顾到了：`reset_for_refill` 在清零行的同时把 steps 还原到已提交值，`rebuild_audio_history` 在 KV 压力重预填充后用 `output_rows` 重放掩码。

概念：重复惩罚 (Repetition Penalty) 与“出现掩码”

它是什么 → 重复惩罚是采样时的一种控制：对“已经生成过的 token”降低其再次被选中的概率，避免模型复读机式重复。这里作用在音频码上。

为什么需要它 → TTS 解码若不抑制重复，容易卡在某个音节反复输出。但“记住每个请求生成过哪些码”本身是个会随时间增长、形状不定的历史——对 CUDA Graph 不友好。

在这个 PR 里怎么用 → 本 PR 把历史表示成固定形状的布尔掩码

`audio_token_presence[行, 通道, 词表]`：生成过就置 True。惩罚时只需一次

`torch.where` 在掩码为真处改写 logits，O(1) 索引、定形、可与图化路径共存。

类比：像给每个客人发一张“已点过的菜”划勾表（格子固定），而不是不断加长的小纸条。

概念：Logits / Temperature / Top-p / Top-k（采样基础）

它是什么 → 模型每步输出一个覆盖整个词表的分数向量叫 logits。temperature 调整分布的“尖锐/平缓”，top-k 只在分数最高的 k 个里采样，top-p 只在累计概率达到 p 的最小集合里采样。

为什么需要它 → 纯取最大（贪心）会让语音呆板；纯随机又会跑偏。这些参数是“可控随机性”的旋钮。

在这个 PR 里怎么用 → 这些参数是请求级静态的，所以 #745/#759 把它们作为 [P] 池子张量，在“拿行”时写一次、每步 gather；重复惩罚则在 temperature 之前、用 fp32 作用于 logits。

类比：像调音台上每个客人有一组固定旋钮，开场设好，演出中按号取用即可。

我能学到什么 / 延伸思考

- “快慢路径分离 + 便宜的标志位决定走哪条”是兼顾性能与功能的经典手法：让 99% 的常见请求走极致优化（图化）路径，把昂贵但少见的特性（惩罚）隔离到 eager 慢路径。
- 用“定形掩码”替换“变长历史”是迁就量化/图化硬件约束的通用技巧——空间换“可预测的形状”。
- 想深入可了解：PyTorch 高级索引与 `torch.where` 的向量化，以及 CUDA Graph capture 为何排斥数据依赖控制流。

MOSS-TTS-Local async 解码：一步前瞻（默认关闭）

作者 JiaxinD · 已合并 · #758 · 建立在 #745 之上 · 12 文件

PR 概览

- 标题：[Perf] MOSS-TTS-Local v1.5 async decode (one-step lookahead) (#734)
- 编号 / 作者 / 状态：#758 · JiaxinD · 已合并（默认关闭）
- 链接：[pull/758](#)

一句话：在 #745 的状态池之上，给 MOSS-TTS-Local 打开“一步前瞻（one-step lookahead）”的 async 解码——把每步“GPU 发射”和“host 收尾”错开一步并行；同时修复了一个只在 bs=1 停止时才暴露的“停止符被覆盖”的隐蔽 bug。

它解决了什么问题

同步解码每一步是“发射 GPU 计算 → 等结果 → host 收集结果 → 再发下一步”。host 收集（resolve）这段纯 CPU 工作，GPU 在干等。一步前瞻的想法是：第 N 步的 GPU 一发出去，先别等它的 host 收尾，立刻去发射第 N+1 步的 GPU 计算，让第 N 步的 host 收尾和第 N+1 步的 GPU 计算重叠起来。

这个 PR 老实地说明：在当前 MOSS 路径上，因为 #745 已经把 host 端尾巴削得极短（每步约 0.06ms vs GPU 步约 10.4ms），实测吞吐 ON/OFF 在噪声内，无可分辨的加速也无回退，所以它的价值是“先把正确性契约和重叠插槽建好”，等 #757 把 resolve 变重后再前默认开关——一个标准的“两阶段落地”策略。

具体做了什么改动

核心是把解码拆成 `post_decode_launch`（GPU 发射半步）与 `post_decode_resolve`（host 收尾半步）两个钩子，并修调度器计数。

(a) 发射半步：给“已发布的 token id”拍一张设备快照

```
def post_decode_launch(self, result, forward_batch, requests):
    if not requests:
        return None
    rows, end_id = self._run_frame_decode(result, requests)  # 跑 GPU 帧
    next_token_ids = self._row_radix_token_ids(rows, rows[:, 0], end_id)
    result.next_token_ids = next_token_ids  # 发布, 给“
    return next_token_ids.clone()  # ★ 关键：拍
```

逐行讲（这是全 PR 最“烧脑”的地方）：

- `_run_frame_decode` 在 GPU 上算出这一帧的 `rows` 和 `end_id`；

- `next_token_ids` 由 GPU radix 哈希算出 (`rows[:,0]` 是文本通道)，纯设备操作、无 host 同步；
- `result.next_token_ids = ...` 把 id 发布出去，好让下一步的 `prepare_for_decode` 拿去拼自己的 `input_ids` (host 收尾被推迟了一步)；
- `return next_token_ids.clone()` 是“承重墙”：base runner 会把 `result.next_token_ids` 别名到 `schedule_batch.output_ids`，而下一步会原地覆盖这个张量。在前瞻下，下一步的发射跑在这一步的滞后 resolve 之前——于是没有快照的话，发布出去的“停止符 (`== end_id`)”在 resolve 拿到它之前就被覆盖没了：bs=1 的 EOS 完成丢失，请求一路跑到 4096 帧上限。`.clone()` 做一份 GPU→GPU 的私有快照（无 host 同步）来扛住这次覆盖。

(b) 收尾半步：把快照换回去（指针交换）

```
def post_decode_resolve(self, launch_buf, result, forward_batch, schedule_batch):
    if launch_buf is not None and result is not None:
        result.next_token_ids = launch_buf        # 把被覆盖前的干净快照换回来
```

就一行有效逻辑：把 `next_token_ids` 重新指回发射时拍下的干净快照，撤销下一步的原地覆盖，于是共享的 `_finalize` 尾巴能读到真正的停止符、EOS 完成照常触发。注意这是指针交换、不是拷贝。

(c) 调度器：补回 `forward_ct`，并丢弃“撤回的陈旧请求”

```
# OmniScheduler 重写了 run_batch，绕过了上游自增 forward_ct 的地方；
# 不补的话 forward_ct 恒为 0，SGLANG_TEST_RETRACT 会每步触发。
self.forward_ct = getattr(self, "forward_ct", 0) + 1    # 每次真实前向 +1 (resolve)

# 撤回的请求(KV 已释放)仍在“滞后 batch”里，必须像“上一步已完成”一样丢掉，避免二次释放 KV
pre_finished = [r.finished() or bool(getattr(r, "is_retracted", False)) for r in batch.reqs]
skip_rids = {batch.reqs[i].rid for i, was in enumerate(pre_finished) if was}
```

前瞻把“发射”和“收尾”拉开一步，于是出现两个原本不会有的竞态：① 调度器的前向计数器要精确（每次真实前向 +1，resolve 不前向所以不计），否则撤回测试每步误触发；② 一个在某步被“撤回 (retraction)”的请求，其 KV 已经释放，但它仍残留在“发射时就建好”的滞后 batch 里——若不丢弃，再跑一遍就会二次释放 KV 触发崩溃。所以 resolve 路径和同步快路径都改成“丢弃 已完成 或 已撤回”的请求。

(d) CLI 门控：从单一 Higgs 工厂扩成集合

```
_ASYNC_DECODE_FACTORIES = frozenset({
    "sglang_omni.models.higgs_tts.stages.create_sglang_tts_engine_executor",
    "sglang_omni.models.moss_tts_local.stages.create_sglang_tts_engine_executor",
})
```

把 `--async-decode` 支持的工厂从“只有 Higgs”改成一个 `frozenset`（现在含 MOSS-TTS-Local），并更新报错文案。MOSS 路径默认仍 OFF，待后续 Phase-3 再翻开关。

概念：Async Decode / 一步前瞻 (One-step Lookahead)

它是什么 → 把解码的每一步拆成两半：“launch（在 GPU 上发射这步的计算）”和“resolve（在 CPU 上把上一步结果收集成可用状态）”。引擎先发射第 N+1 步，再去做第 N 步的 resolve，让两者重叠。

为什么需要它 → GPU 算得快，但每步之间 CPU 要做一堆“收尾”杂活（收集 token、更新状态、判断停止），这段 CPU 时间里 GPU 闲着。重叠起来就能把 CPU 尾巴藏到 GPU 计算背后。

在这个 PR 里怎么用 → 本 PR 用 `post_decode_launch` / `post_decode_resolve` 实现这套错位；难点在于“发布出去的张量会被下一步原地覆盖”，所以用一次 GPU 端 `.clone()` 快照守住 `bs=1` 的停止符。当前是“正确性契约 + 预留的重叠插槽”，性能收益留待 resolve 变重后兑现。

· 类比：像厨师把菜下锅(launch)后不守着，转身去摆上一道菜的盘(resolve)；但要先拍张照记住这锅菜该几分熟，免得下一道菜把灶台占了、火候记错。

概念：Retraction (请求撤回) 与双重释放

它是什么 → 在显存吃紧时，调度器可以把某个正在解码的请求“撤回”：释放它的 KV cache、把它退回等待队列，等显存宽裕再重新预填充恢复。

为什么需要它 → 连续批处理下并发请求数是动态的，撤回是引擎在 OOM 边缘“弹性让位”的安全阀。

在这个 PR 里怎么用 → 一步前瞻让 batch 在“撤回发生前”就被建好，于是被撤回的请求会以“陈旧成员”身份残留在滞后 batch 里；本 PR 在 resolve 和快路径都加上 `is_retracted` 过滤，避免对已释放的 KV 二次释放（原本只过滤 finished）。

· 类比：像把已经离场的客人从“上一轮就排好的上菜清单”里划掉，免得给空座位再上一份、还重复收走餐具。

我能学到什么 / 延伸思考

- 诚实的性能叙事：本 PR 明确写“当前无可分辨加速”，把机制正确性与吞吐收益分开陈述，并解释为何留作“预留插槽”——这是工程严谨度的范例，避免夸大。
- “别名 + 原地覆盖”是高性能代码里的隐形地雷：当你把一个张量发布给下游，下游又会复用同一块内存时，跨越异步边界就需要快照。这个 bug 只在 `bs=1+EOS+Lookahead` 同时成立时才现形，提醒我们边界条件测试的重要性。
- 想深入了解：CUDA stream 与异步执行、张量别名 (aliasing) / 视图，以及调度器的 forward 计数与撤回机制。

优化 Higgs 参考音频缓存键（内容寻址）

作者 Ratish1 · 已合并 · #713 · 3 文件

PR 概览

- 标题：[Higgs]: Optimize Higgs ref-audio cache keying
- 编号 / 作者 / 状态：#713 · Ratish1 · 已合并
- 链接：[pull/713](#)

一句话：Higgs TTS 做“声音克隆”要先把参考音频编码成参考码（reference codes）并缓存；本 PR 把缓存键从“音频来源字符串”改成“解码后波形的内容哈希”，于是指向同一段音频的不同 URL 也能安全命中缓存。

它解决了什么问题

Higgs 是声音克隆型 TTS：你给一段“参考音频”，它模仿那个音色说出目标文本。把参考音频用音频编码器编成“参考码”很贵，所以要缓存。

老的缓存键基于来源：本地路径/base64 还好，但 URL 直接拿 URL 字符串当键就有两个毛病——① 同一段音频挂在两个不同 URL 上，会被当成两份、重复做昂贵编码；② URL 字符串本身不可靠（同一 URL 内容可能变）。

缓存键应该指向音频解码后的内容，而不是你从哪里引用它

具体做了什么改动

核心文件 `models/higgs_tts/stages.py`，把“一个缓存键”拆成两个职责不同的键。

```
def _reference_code_cache_key_from_waveform(waveform, sample_rate):  
    """音频解码/重采样之后，用‘消耗给编码器的波形内容’做参考码缓存键。"""  
    wav = waveform.detach().cpu().contiguous().float()  
    meta = f"sr:{int(sample_rate)}|shape:{tuple(wav.shape)}"  
    return f"waveform:{meta}:{hash_bytes(wav.numpy().tobytes())}"
```

逐块解释：

- source key（来源键）：仍用来查“波形缓存”（避免对本地/base64 重复解码）。URL 不进这个缓存。
- content key（内容键）：上面这个函数——把已经解码、重采样到 24kHz 的波形张量按字节哈希，带上采样率和形状元信息。它才是“参考码缓存”的键。

于是流程变成：先按来源查波形缓存；解码出 24kHz 波形后，用波形内容算 content key，再去查/写昂贵的“参考码缓存”。两个不同 URL 只要解码出的波形一样，content key 就一样，安全复用参考码。配套把状态字段

`reference_cache_key` 改名为更准确的 `reference_code_cache_key`，并加了 URL 复用、内容键只算一次等单测。

概念：声音克隆中的“参考音频 → 参考码 (codes)”

它是什么 → TTS 里的音频编码器 (codec / audio tokenizer) 把一段连续波形压成一串离散的“音频 token (codes)”，模型在这串 codes 的条件下生成同音色的新语音。

为什么需要它 → 直接在原始波形上做条件计算又慢又难；把音频离散成 codes，就能像处理文本 token 一样喂给 Transformer，还能缓存复用。

在这个 PR 里怎么用 → 本 PR 优化的正是“参考码缓存”的命中率：把键绑定到解码后的波形内容，让相同音频（即便来源不同）只编码一次。

类比：像把一段录音先转成乐谱(codes)，谁来要这段都给同一份乐谱，而不是每次重新扒谱。

概念：内容寻址缓存 (Content-addressed Cache)

它是什么 → 用“数据内容的哈希”当缓存键，而不是用“数据的名字/位置/URL”。内容相同 → 键相同 → 命中。

为什么需要它 → 用名字做键会漏掉“同物异名”（两个 URL 同一段音频）和误中“同名异物”（同一 URL 内容变了）。内容寻址从根本上避免这两类错配。

在这个 PR 里怎么用 → 本 PR 把参考码缓存改成内容寻址（波形字节哈希），同时保留来源键用于更便宜的“波形解码缓存”——两级缓存各司其职。

类比：像 Git 用文件内容的哈希做对象 id：内容一样就是同一个对象，叫什么名字无所谓。

我能学到什么 / 延伸思考

- “缓存键要绑定语义内容，而非引用方式”是一条可直接复用的缓存设计准则：并发性能/正确性问题常常出在键选错了。
- 把一个键拆成两个职责不同的键（便宜的来源键 vs 昂贵的内容键）是“分层缓存”的实用模式——便宜的先挡一层，贵的用更准的键。
- 想深入可了解：音频 codec / 神经音频 tokenizer、重采样 (resampling)，以及哈希在缓存与去重中的应用。

打磨 Higgs decode-mode CLI

作者 Ratish1 · 已合并 · #700 · CLI/配置层

PR 概览

#700 · Ratish1 · 已合并 · [pull/700](#)。把 Higgs 暴露给用户的 `--async-decode default|on|off` 旧开关，换成更直白的 `--decode-mode async|sync`

动机与改动

Higgs 现在默认就用 async 解码，旧的三态开关 `--async-decode default|on|off` 既泄露了实现术语、又让“不传参时到底走哪个默认”难以推理。改动：

- 用 `--decode-mode async|sync` 取代旧标志；不传 = 用流水线默认，传了才覆盖。内部仍映射到原来的 `enable_async_decode` 工厂参数。
- 高级调参标志改名为 `--async-lookahead-min-batch-size`；当与 `--decode-mode sync` 同用时 CLI 直接报错（sync 模式根本不用前瞻阈值）。
- 重写该 CLI 的单测，覆盖默认 async、显式覆盖、省略、阈值校验、不支持的配置、以及旧标志已删除等场景。

纯 CLI/配置层改动，不碰调度器或模型执行路径。

概念：CLI 作为“契约表面（surface）”的设计

它是什么 → 命令行参数是用户与系统之间的约定。好的 CLI 应表达“用户意图”，而不是暴露内部实现细节或模糊的默认。

为什么需要它 → `--async-decode default|on|off` 把内部实现词（async）和三态默认糊在一起，难懂；`--decode-mode async|sync` 直接表达“我要哪种解码模式”，省略即默认。

在这个 PR 里怎么用 → 本 PR 还顺手做了“非法组合早失败”（sync + 前瞻阈值直接报错），把错误尽早、清晰地反馈给用户。

类比：像把开关上的标签从“模式A/B/默认”改成直接写“快/慢”，谁都看得懂。

我能学到什么 / 延伸思考

- 面向意图、隐藏实现。非法组合早报错是 CLI/API 设计的通用好习惯，可直接套用到自己的工具。

适配 Blackwell (SM100+) GPU

作者 zhaochenyang20 · 已合并 · #763 · 新增 gpu_compat.py

PR 概览

#763 · zhaochenyang20 · 已合并 · [pull/763](#)。让 sglang-omni 在 NVIDIA Blackwell (SM 100+) 新卡上跑起来：按 GPU 的 SM 版本自动设置 FlashInfer/注意力后端的兼容兜底。

动机与改动

新增 `utils/gpu_compat.py` (+148)，核心是探测每张可见 GPU 的“计算能力 (compute capability / SM 版本)”，并据此自动注入兼容性环境变量与后端选择：

```
def get_visible_gpu_sm_version(logical_gpu_id, env=None):
    """返回某张可见 GPU 的 SM 版本, 如 90(Hopper)、100(Blackwell)。"""
    capability = _get_compute_capability(logical_gpu_id, env or os.environ)
    if capability is None:
        return None
    major, minor = capability
    return major * 10 + minor          # 例如 (10,0) -> 100

def get_gpu_compat_env_defaults(env=None):
    """SM>=100(Blackwell) 时, 默认打开 FlashInfer 的 CUDA norm 兼容开关。"""
    source_env = env or os.environ
    if source_env.get("FLASHINFER_USE_CUDA_NORM") is not None:
        return {}                    # 用户已显式设置就不覆盖
    if not visible_gpus_need_flashinfer_cuda_norm(source_env):
        return {}
    return {"FLASHINFER_USE_CUDA_NORM": "1"}
```

- 用 NVML (pynvml) 优先、PyTorch 兜底两条路探测 SM 版本，兼容 `CUDA_VISIBLE_DEVICES` 的设备重映射。
- SM $\geq$ 100 (Blackwell) 时自动设 `FLASHINFER_USE_CUDA_NORM=1`；Qwen3-ASR 路径在 SM $\geq$ 100 时把多模态注意力后端切到 `triton_attn`。
- 在 server 启动 (`launcher.py`) 和各 stage worker 进程 (`stage_workers.py`) spawn 时都应用这些默认，保证子进程也带上正确环境。
- 顺手清理了 Qwen3-ASR 的 stages：把延迟导入提到模块顶层、用 AutoTokenizer/AutoFeatureExtractor 替代 AutoProcessor。

概念：GPU 计算能力 / SM 版本与 Attention Backend

它是什么 → 每代 NVIDIA GPU 有个“计算能力”号 (compute capability)，常折成 SM 版本：如 Ampere=80、Hopper=90、Blackwell=100。它决定了哪些算子实现/特性可用。Attention Backend 是注意力计算的具体实现库 (FlashInfer、Triton 等)。

为什么需要它 → 新架构刚出时，高性能算子库 (如 FlashInfer) 对它的支持可能不全或有 bug。引擎需要按硬件代际自动选择“能正确跑的后端/开关”，否则新卡直接报错。

在这个 PR 里怎么用 → 本 PR 探测到 Blackwell(SM100+) 就自动启用 FlashInfer 的 CUDA norm 兜底、并把 ASR 的多模态注意力换成 triton 后端，让代码在新卡上开箱即用、又不影响老卡 (默认不动)。

类比：像新型号插座刚上市，万能转换头会自动识别制式并切换到兼容档，老插座照旧。

我能学到什么 / 延伸思考

- “按硬件代际自动降级/切换后端”是推理基础设施的常见兼容性策略：探测能力 + 仅在需要时注入默认 + 不覆盖用户显式设置，是稳妥的兜底写法。

升级钉死的 SGLang 后端 + 适配多模型组件

作者 Hayden727 · 已合并 · #698 · 30 文件 · Closes #658

PR 概览

#698 · Hayden727 · 已合并 · [pull/698](#) (Closes #658)。升级钉死 (pinned) 的 SGLang 后端版本，并把众多模型组件适配到新版 API。

动机与改动

sglang-omni 把底层 SGLang 当依赖钉在某个版本。底层升级能拿到新特性/修复（如 CUDA13 栈、qwen3-vl 上游支持），但也意味着要跟着改一堆调用点。本 PR 跨 30 个文件：

- 依赖与 CI：`pyproject.toml` 调整版本约束、新增 `install_flashinfer_jit_cache.sh`、更新各 CI workflow 与文档安装说明。
- 模型适配：精简 `_sglang_qwen3_vl_patches.py`（-104 行，说明上游已原生支持，少打补丁）、调整 `qwen3_omni` 的 `thinker/talker/image_encoder`、`moss_tts`、`ming_omni`、`fishaudio_s2_pro` 等多个多模态模型组件到新 API。
- runner 层：`model_worker.py`、`sglang_model_runner.py` 跟随上游接口变化调整。

这类 PR 不引入新功能，但是“站在巨人肩膀上”的必要维护：上游 SGLang 持续优化，omni 分支要定期 rebase 才能持续受益。注意 #764（CI 镜像换阿里云）正是这次升级引入 CUDA13 依赖后，为修 CI 取包 404 的连带修复。

概念：依赖钉版 (Pinning) 与上游同步 (Rebase onto upstream)

它是什么 → “钉版”是把依赖锁定在确切版本，保证构建可复现。“上游同步”是周期性地把上游新版本合进来并适配。

为什么需要它 → 锁死能稳定，但会逐渐落后、错过性能与修复；定期升级能持续受益，代价是适配上游的接口变更 (breaking changes)。这是稳定性与新鲜度之间的权衡。

在这个 PR 里怎么用 → 本 PR 是一次集中的上游同步：升级 SGLang 版本，同时把 qwen3-vl、qwen3-omni、moss/ming/fish 等模型组件改到新 API；并因新栈依赖 CUDA13 包，连带触发 #764 换镜像。

类比：像 App 跟随系统大版本升级：能用新能力，但要把被改动的系统接口逐一适配。

我能学到什么 / 延伸思考

- 衍生分支要定期 rebase 上游，并把适配集中成一个可审阅的 PR 是依赖管理的现实功课；一次升级常引发连锁（如 CI 取包源），要顺看修。

PR #764 · 基础设施

CI PyPI 镜像切换到阿里云

作者 Ratish1 · 已合并 · [#764](#) · CI 配置



PR 概览

#764 · Ratish1 · 已合并 · [pull/764](#)。把 Omni CI 默认的 uv PyPI 镜像从清华源切到阿里云源。



动机与改动

承接 #698：CUDA13 栈现在依赖 `mooncake-transfer-engine-cuda13>=0.3.10`。清华 PyPI 镜像对该包返回 404，于是 CI 的 `setup - omni venv` 在 `uv pip install -e .` 阶段就失败、所有测试任务都跑不起来。阿里云镜像有 CUDA13 中继 wheel，切过去即可。同时让本地 CI 环境助手与 GitHub Actions 的 setup 路径保持一致。

我能学到什么 / 延伸思考

- 镜像源不是“无所谓的小事”：依赖升级后，包是否在你用的镜像上有同步，会直接决定 CI 能否启动。基础设施的“取包源”要和依赖版本一起被当成一等公民管理。

本日报由对 `sgl-project/sglang-omni` 过去 24h merged PR 的源码 diff 逐一研读后生成，面向“推理基础设施小白”做了概念补全。代码片段忠实摘自各 PR 的实际 diff（仅取新增行、去掉前导 +）。术语解释为帮助理解所作的通俗化表述，权威细节请以各 PR 源码与设计文档为准。

生成日期 2026-06-13 · 主题聚焦 Omni / 多模态 / 多阶段 TTS 推理架构。